Delay propagation at the Brussels North–South junction

Data, state of the art, and Phase 0 results

TReC Causality Team

August 26, 2026

Where this is going

Objective. Use the SUMO microsimulation as an **interventional ground truth** to validate causal-discovery methods for delay propagation at the junction, then apply the validated method to the real Infrabel record — and read the gap between them as the *dispatcher effect*.

- Observational railway data records *what* happened, never *why* a train was held. No dispatcher intervention logs $\Rightarrow$ treatment-effect methods cannot be identified from real data alone.
- SUMO can run the counterfactual. That is the wedge.

This deck covers **Part I** the data, **Part II** the state of the art, and **Part III** Phase 0 — establishing the observational facts and a performance floor.

Outline

Part I — The data

Source and scale

Infrabel open data, dataset `stiptheid-gegevens-maandelijksebestanden` (151 monthly files). We use **January 2025**.

Raw file	329.5 MB
Rows	1,965,004
Columns	21, comma-separated
Full year	≈23M rows / 4 GB

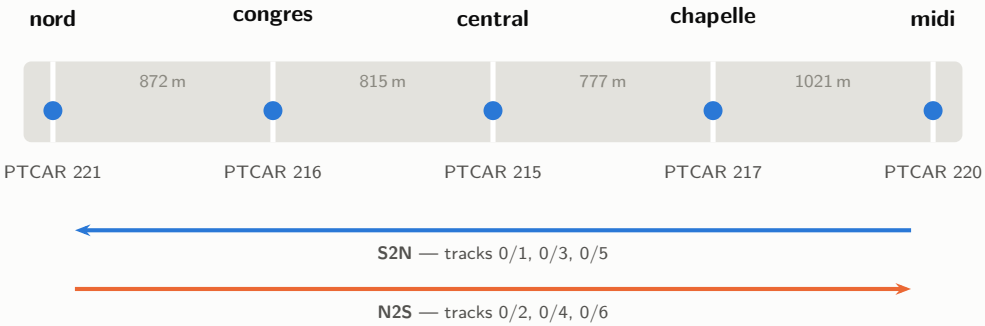
Gotchas found in the feed:

- The *manifest* is `;`-separated, the *data* is `,`
- `PTCAR_LG_NM_NL` carries the **Dutch** name only
- Hours are **not zero-padded** (`0:01:11`, `9:57:00`) — string comparison of times is wrong
- Open data says `PTCAR_NO`; the gold layer says `PTCAR_ID`

Delay columns are signed seconds (negative = early), precomputed as `DELAY_ARR` / `DELAY_DEP`. `THOP1_COD` marks stop type: `=` commercial stop, `D` pass-through, `P` origin/terminus.

What “the junction” is

The **Brussels North–South connection** (*Jonction Nord–Midi / Noord-Zuidverbinding*): a six-track tunnel running under central Brussels that links Bruxelles-Nord to Bruxelles-Midi, with three intermediate stops. **Every train crossing Belgium north-to-south passes through it.**



≈3.5 km end to end · hops of 777–1021 m
968 traversals per day, both directions

Six tracks, **three per direction**, each strictly one-directional
A train holds *one* track for the whole crossing

Identifying the junction

`PTCAR_NO` uses the same numbering as `STATIONS` in `core/simulation/build_hard.py`, confirmed against the Infrabel reference dataset `operationele-punten-van-het-netwerk` (1,359 points with coordinates).

PTCAR	Symbolic	Name (FR)	Data name (NL)	lat, lon
215	FBCL	BRUXELLES-CENTRAL	BRUSSEL-CENTRAAL	50.845, 4.357
216	FBCO	BRUXELLES-CONGRES	BRUSSEL-CONGRES	50.852, 4.362
217	FBCK	BRUXELLES-CHAPELLE	BRUSSEL-KAPELLEKERK	50.841, 4.348
220	FBMZ	BRUXELLES-MIDI	BRUSSEL-ZUID	50.836, 4.336
221	FBN	BRUXELLES-NORD	BRUSSEL-NOORD	50.860, 4.361

216 and 217 are classified *stop in open track*, not stations — they are 95–98% pass-through in the data, which matters later.

Transformation pipeline — exactly what was done

- **Step 1** `extract_junction.py` — streams the raw file once (≈7 s), keeps the 5 junction PTCARs. Retains 7.77% of rows.
- **Step 2** keep only trains passing through *both* Nord and Midi, i.e. that actually traverse. Drops 2,616 single-station touches (2,532 at Midi).
- **Step 3** `build_traversals.py` — pivot the 5 points into `p1..p5` **ordered along direction of travel**, add `direction`, `tunnel_track`, `entry_delay`, `exit_delay`, `delay_gained`.

Reading a traversal row — what p1…p5 mean

After the pivot, each row is *one train crossing the junction once*, and the five measurement points become columns p1_* through p5_*.

p1…p5 are positions *along travel*, not fixed stations

p1 is always the point the train **enters** at and p5 the point it **leaves** at — so p1 is Nord for an N2S train but Midi for an S2N train. This is deliberate: it makes p1_delay_arr … p5_delay_arr a single comparable sequence "entry → exit" regardless of direction.

train_no=1945 · IC 06-1: BRUSSELS AIRPORT → TOURNAI · direction=N2S · tunnel_track=0/4

	point	arrival	delay_arr	departure	delay_dep	stop_type
p1	nord (221)	23:58:50	−9	0:01:11	+11	= stop
p2	congres (216)	0:03:15	+15	0:03:15	+15	D pass-through
p3	central (215)	0:04:50	−9	0:06:16	+16	= stop
p4	chapelle (217)	0:07:20	−40	0:07:20	−40	D pass-through
p5	midi (220)	0:09:25	−34	*(blank)*	*(blank)*	*(blank)*

entry_delay = p1_delay_dep = +11 s exit_delay = p5_delay_arr = −34 s **delay_gained = −45 s**

Note this row also crosses midnight (23:58 → 00:09) while date stays on 01JAN — one of 59 such traversals. And it shows the blank pattern explained next.

Structural facts that shaped everything downstream

The structure is completely regular

- 29,997 of 32,613 trips give *exactly* 5 rows
- Zero fell into "unexpected order" or "not five points"
- Only two row orders occur, evenly split: 220→217→215→216→221 (S2N, 14,964) and its reverse (N2S, 15,033)
- **Direction is recoverable from row order alone**

Tunnel tracks are one-directional

Each traversal holds a *single* tunnel track across 215/216/217 — verified: distinct-track-count per trip is 1 in all 29,997 cases.

Track	Direction	Traversals
0/1	S2N	5,673
0/2	N2S	5,890
0/3	S2N	5,502
0/4	N2S	5,767
0/5	S2N	3,789
0/6	N2S	3,376

Odd tracks southbound-to-north, even tracks north-to-south. Three per direction — the six-track N–S tunnel.

What a “blank” is, and where they occur

A blank is simply an empty cell in the CSV — Infrabel recorded no value for that field on that row. Each measurement point has *two* sides, an arrival and a departure, and either can be blank independently.

Across the 29,997 traversals, blanks appear at **exactly two of the ten delay columns**:

Column	Side	Blank rows	
`p1_delay_arr`	arrival at the **entry** point	4,741	
`p5_delay_dep`	departure at the **exit** point	4,789	
`p1_delay_dep`, `p2..p4`, `p5_delay_arr`	(all others)	**0**	**never blank**

They come as a **bundle**: on a p5-blank row, `THOP1_COD`, `PLANNED_TIME_DEP`, `REAL_TIME_DEP` and `LINE_NO_DEP` are *all* empty together (4,701 of 4,720) while the arrival fields are fully populated. So it is one coherent “no onward record at this point” class, heavily concentrated at Midi (4,336 of 4,720).

Why the blanks do not hurt us — and what we must not claim

The convenient part

`delay_gained` = `p5_delay_arr` − `p1_delay_dep` — the arrival at the exit minus the departure at the entry. Those are precisely the two sides that are **never blank**. **So `delay_gained` is present for all 29,997 traversals**: no imputation, no row-dropping, for the headline propagation variable.

The tempting-but-wrong explanation

It looks like "the train terminated there, so there is no departure." **That reading does not survive checking:**

- Some of these trains *do* end at Brussel-Zuid (`IC 35: ROTTERDAM CENTRAAL -> BRUSSEL-ZUID`).
- But others plainly continue — our worked example `IC 06-1 -> TOURNAI`, and `IC 29: DE PANNE -> LEUVEN`.
- And the feed *has* an origin/terminus code, `THOP1_COD = P` — **no Midi row ever carries it**.

Treat it as a **recording gap of unconfirmed cause**. Worth asking Infrabel.

Part II — State of the art

Three predictive paradigms

1. **Spatio-temporal GNNs.** Heterogeneous graphs (SAGE-Het) with train, station and time-slot node types; graph attention (GATv2) to weight propagation dynamically. Li et al. 2024; Nguyen & Li 2025.
2. **Simulation-driven imitation learning.** Reframes forecasting as stochastic simulation of state transitions; DCIL extends DAgger with drift correction to fight covariate shift in rollouts. Elliker et al. 2025 — *on Infrabel data.*
3. **Tree ensembles + conformal prediction.** GBDTs on tabular history, paired with distribution-free prediction intervals for dispatching support.

The benchmark that matters for us

RIDE (arXiv 2606.05070) — open benchmark on the **Belgian** network: 94.5M train events, 3.6M journeys, **35.7M weather records**, 2023–2025, keyed to operational points. Weather is our largest missing input and RIDE is aligned to our identifiers.

Causal discovery and causal ML

- **Reconstructing the propagation network.** Constraint-based discovery (PC/FCI) and PCMCI on lagged multivariate series; Hawkes–Granger processes for event-triggered propagation.
- **"What-if" dispatching.** Framing a hold-or-not decision as a treatment and estimating ITE/ATE, to separate deliberate dispatcher action from natural propagation.
- **Unified GNN + CML + conformal** frameworks.

Verification matters — claims checked individually

- **"GNNs consistently achieve the lowest MAE" is inverted** for the paper it rests on: Nguyen & Li report *"the much simpler Persistence baseline achieves the best scores on MAE and RMSE."* Their GATv2 wins on *precision*.
- **PCMCI-on-railway** and **conformal-prediction-on-train-delay**: no arXiv evidence found. Good ideas, not established practice.
- A **Scientific Reports "unified GNN+CML+CP" paper** turns out not to be a delay paper at all: target is *total Vehicle KM*, $n = 288$ line-level aggregates, and the word "delay" appears 5 times, all in related work.

What actually applies at our scale

Our unit is a **5-node, 6-track junction**, one month, 29,997 traversals.

Do now	Persistence baseline; conformal intervals (heavy tails); GBDT on the tabular traversal table
Tested, adds nothing	Weather at junction scale: −1.7% MAE, six features (Step 7). RIDE's 2023–2025 span is the way to test it properly
Overkill	ST-GNN on 5 nodes — no topology to learn. Same objection applies to Dekker's diffusion model
Blocked	Causal ML for dispatching — needs intervention logs that the Infrabel feed does not contain

One convergence worth noting: Nguyen's explainability analysis finds the model keys on local congestion to spot fragile states where *inter-train headway conflicts* propagate delay — the same mechanism as Li's 20-minute threshold, and the same one we measure directly in Phase 0.

Part III — Phase 0

Observational facts, and a floor to beat

Phase 0 — the steps

1. Build the **leader/follower pair table** on shared tunnel track
2. Define the **prediction task** and a temporal split
3. **Baselines**: zero, persistence, mean-gain
4. A **feature model** that must beat them
5. Test **Dekker's exponential-decay claim**
6. Facts sheet and figures
7. Add **weather** and test whether it earns its place

In one sentence

Given how late a train is **entering** the junction, plus who is ahead of it and how close, predict how late it will be **leaving**. Next slide makes this precise.

The prediction task, precisely

One row = one train crossing the junction, paired with the train ahead of it on the same tunnel track.

Output (continuous, signed seconds)

`follower_exit_delay` = `p5_delay_arr`

the train's delay on *arrival* at the far side

Prediction moment

the instant it *departs* the entry point `p1`.
Every feature must be known then.

Baselines predict the same target with no model:

`zero`: 0 · `persistence`: `entry_delay` (the junction changes nothing) · `mean-gain`: `entry_delay` + 26.9 s

Input — seven features

`entry_delay`	its own delay leaving `p1`
`hour`	hour of entry
`tunnel_track`	which of the six tracks
`direction`	N2S or S2N
`service`	IC / L / P / EURST
interaction block	
`headway_s`	gap to the train ahead
`leader_entry_delay`	its delay when *it* entered

Unit of observation: the *pairs* table, not the traversals table. A pair needs a predecessor, so the first traversal of each (`date`, `direction`, `track`) group has none — 29,997 → **29,685 rows**, 312 dropped. **Split:** temporal, train days 1–24 (22,817), test days 25–31 (6,868); no shuffling. No distribution shift (means 163.4 vs 159.5 s, p90 406 both).

Step 1 — pairs, and a leakage trap

A *pair* is two consecutive traversals sharing `(date, direction, tunnel_track)`. **29,685 pairs**, median headway **5.3 min**. (312 traversals have no predecessor in their group and drop out.)

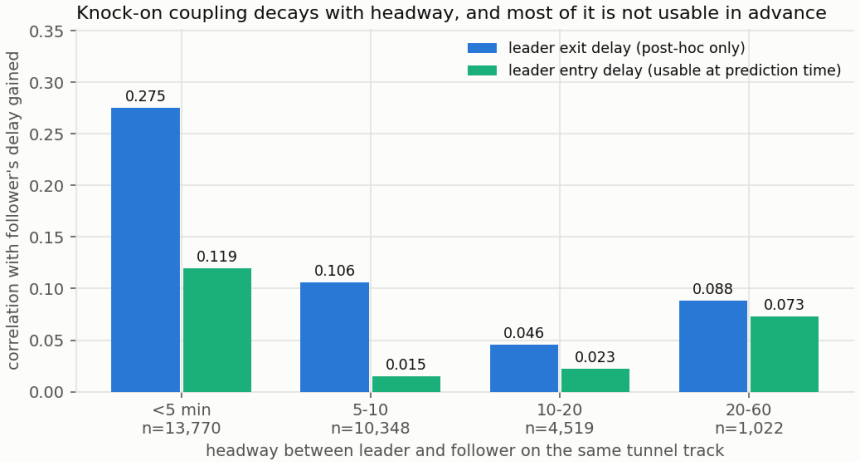
The trap

Median headway (5.3 min) is *shorter* than median traversal time (8.8 min), so **the leader is usually still inside the junction** when the follower enters. `leader_exit_delay` is therefore **not knowable at prediction time**. Models use `leader_entry_delay` instead.

This costs most of the apparent signal — and that is the honest picture:

Headway	r(leader **exit**, follower gain)	r(leader **entry**, follower gain)
<5 min	0.275	**0.119**
5–10 min	0.106	0.015
10–20 min	0.046	0.023
20–60 min	0.088	0.073

Step 1 — knock-on decays with headway



Coupling decays monotonically and is essentially gone by 10–20 minutes — independently reproducing Li et al.'s ≈20-minute interaction horizon on Brussels data. The junction operates deep inside the interacting regime: 96% of consecutive same-track pairs are under 20 minutes.

Steps 2–4 — baselines and the feature model

	regression		*derived*: >300 s event	
Model	MAE (s)	RMSE (s)	Precision	Recall
zero	171.4	380.3	—	0.000
persistence	**60.2**	**99.4**	0.960	0.653
mean-gain	63.1	94.1	0.919	0.694
GBDT (no interaction)	66.1	143.2	0.931	0.698
GBDT (+ headway/leader)	**57.4**	127.1	0.937	0.710

1. **Persistence is hard to beat.** A GBDT with entry delay, hour, track, direction and service but *no interaction features* **loses to it by 9.7%**.
2. **Only the headway/leader block beats the baseline** — worth +8.63 s MAE (13.1%), the entire source of the win. *The one feature block with a causal story is the one carrying signal.*
3. Margin is stable under rolling origin (+6.1%, +4.6%) — not a one-week fluke.

Precision/recall above are *not* a second model — see next slide.

On the “>5 minutes late” event — we are not doing classification

Every model in Phase 0 is a regressor. The precision and recall columns are the *regression output thresholded post-hoc* at 300 s (`common.py:scores()`: `y_pred > threshold`). There is no classifier.

Why report it. The operational question is not “how many seconds” but “will this train be more than 5 minutes late” — and it is the metric on which Nguyen & Li’s GATv2 wins while *losing* on MAE.

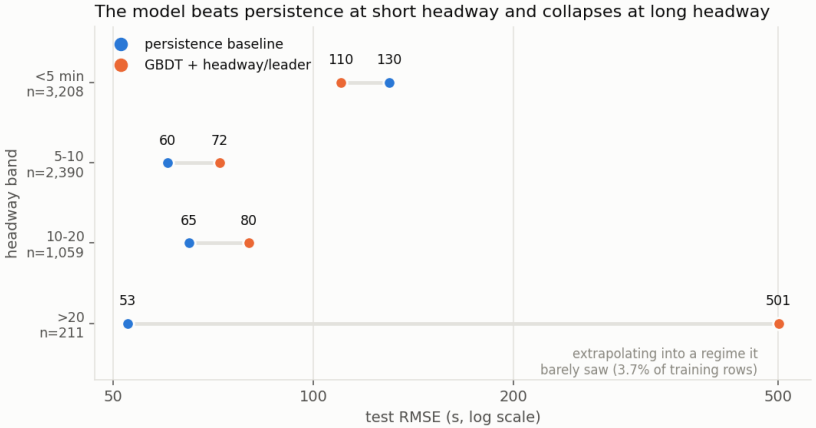
Why it is under-served. A squared-error regressor does not optimise that event:

Approach	Precision	Recall	F1
persistence, thresholded	**0.960**	0.653	0.777
GBDT regressor, thresholded (reported above)	0.937	0.710	0.807
GBDT classifier trained on the event	0.935	**0.743**	**0.828**

Same precision, +3.3 **points of recall**, and the threshold becomes a tunable knob rather than an artifact of the loss. Base rate in test: 14.8%.

Recommendation. Keep regression primary — `delay_gained` is the causal quantity SUMO must reproduce — but if the 5-minute event drives dispatching, **train for it explicitly**.

Where the model breaks — and why



The global RMSE loss (127.1 vs 99.4) is *not* a tail problem. It localises entirely to the >20 min band — 3.7% of training rows — where the model extrapolates badly. **The fix comes from Li et al. 2024**: above ≈20 min there *is* no train interaction, so those features are noise. Gating to persistence above the threshold gives **56.0 MAE / 92.4 RMSE**, beating persistence on both.

Caveat: gating repairs the second fold, not the first. RMSE remains partly unexplained — MAE gain is established, the RMSE

Step 5 — testing Dekker's exponential decay

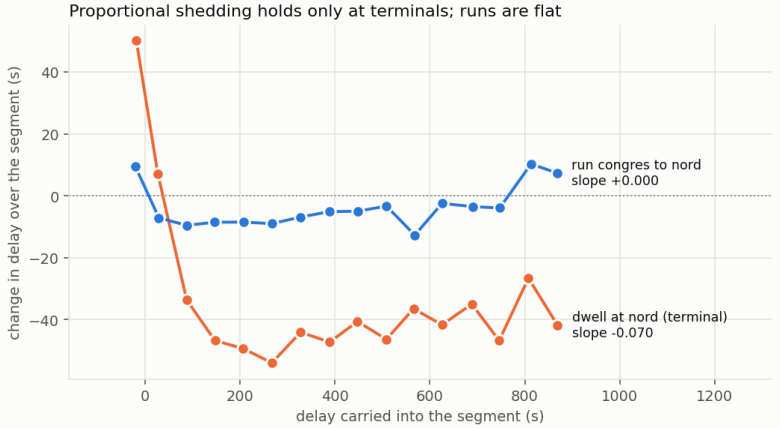
Claim. A station sheds delay in proportion to the delay it carries, $\mathrm{d}D/\mathrm{d}t = -B_i D_i$ — so the change in a train's delay should be *linear* in delay carried, with *negative* slope.

Method. Per direction and segment, regress change-in-delay on delay-carried. Dwell rows restricted to trains that actually stop (`stop_type = '='`) — **Congrès and Chapelle are 95–98% pass-through**, where `arr` = `dep` by construction and the regression would be definitionally zero.

4 of 17 segments consistent · 0 contradict · 13 flat

- **Consistent** — dwell at the two terminals only: Midi (-0.058 S2N, -0.034 N2S), Nord (-0.070 S2N, -0.052 N2S).
- **Flat** — dwell at Central, and *every* run segment ($|\text{slope}| \leq 0.019$), but with nonzero intercepts ($+6.2$, $+8.8$, -17.1, -25.3 s).

Step 5 — additive, not multiplicative

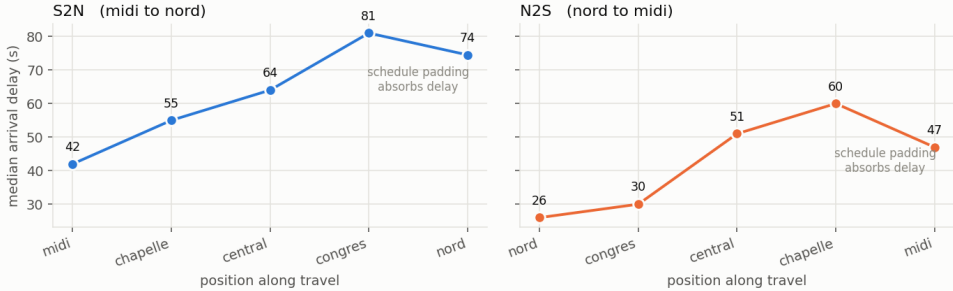


Between points, delay changes by a roughly **constant offset independent of the delay carried**. The diffusion model's core mechanism does *not* operate at junction scale, except at the two big terminals where long dwells give room to recover proportionally.

A useful negative result: a proportional-decay term is the wrong functional form here. The right skeleton is a constant per-segment offset plus a terminal recovery term — and that is a concrete calibration target for SUMO.

The propagation profile

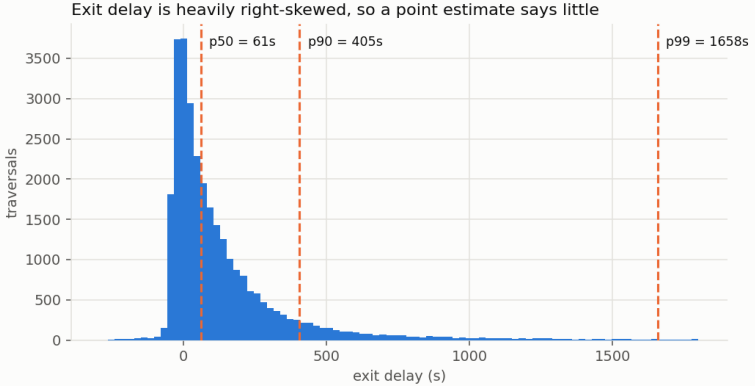
Delay accumulates through the tunnel, then drops at the exit station



Delay accumulates monotonically through the tunnel, then *drops* on the last hop (congrès 81 → nord 74; chapelle 60 → midi 47). Nothing physical removes delay there — it is **recovery margin padded into the final approach**.

Consequence: the p4→p5 hop must be excluded from any propagation fit, or modelled separately and labelled. Otherwise it reads as "the junction heals delay".

Why point forecasts are not enough



Mean 162 s against median 61 s; p99 at 1658 s. Delay gained across the junction has median ≈ 0 but mean $\approx +28$ s — the junction is neutral for a typical train and the average is driven entirely by a heavy right tail.

Any model fit on the mean is fitting the tail. This is the concrete argument for **conformal prediction intervals** rather than point estimates.

Step 7 — weather (a negative result)

Method. Replicated RIDE's own recipe (arXiv 2606.05070): Open-Meteo Historical Weather API, queried per operational point, timezone `Europe/Brussels`, six hourly variables — `temperature_2m`, `rain`, `snowfall`, `relative_humidity_2m`, `wind_speed_10m`, `weather_code` — attached at the **entry point and entry hour**, i.e. the prediction moment. 100% join rate.

January 2025 was not a quiet month: mean 2.8 °C, 200 sub-zero hours, 174 rain hours, 38 snowfall hours.

Model	MAE (s)	RMSE (s)
persistence	60.2	99.4
GBDT (+ headway/leader)	**57.4**	127.1
GBDT (+ weather)	58.4	129.7

The weather block is worth −0.99 s MAE (−1.7%) — six added features, and **slightly worse**. No help in the bad-weather subgroup either (rain >0.5 mm, snowfall, or sub-zero; $n = 855$): 49.2 s MAE with weather against **48.0** without.

And this is the expected answer. The junction is a 3.5 km sheltered tunnel and a train is inside for ≈9 minutes. Weather acts on the *network*; its effect arrives already encoded in `entry_delay`, the model's first feature.

Step 7 — weather is not the confounder

The result that matters

Weather was the most plausible latent common cause behind the leader/follower correlation — and it is not it. Partial correlation of `leader_entry_delay` vs `follower_delay_gained`, headway <5 min:

raw (the 0.119 of Step 1)	+0.1195
controlling for weather	+0.1217
controlling for weather + hour	+0.1151

Essentially unchanged — which strengthens the case that the coupling is a genuine track-occupancy mechanism rather than shared exposure.

The one apparent signal was a single day

Daily snowfall against mean entry delay gives $r = +0.414$ (95% CI $[+0.07, +0.67]$, $n = 31$) — but dropping 09JAN2025, the one heavy snow day, collapses it to $r = -0.007$. Snow days average 138 s entry delay against 132 s on dry days.

This says weather adds nothing to *this* model at *this* scale, not that weather is irrelevant to Belgian rail delay. One month, six snow days, one that matters. Testing it properly needs RIDE's 2023–2025 span.

Phase 0 — conclusions

1. **A floor exists and it is high.** Persistence: MAE 60.2, RMSE 99.4. Feature richness alone does not beat it.
2. **The causal feature block is the only one that pays.** Headway + leader state is the entire margin — arrived at empirically, not assumed.
3. **Knock-on is real, concentrated, and short-range.** Strongest below 5 min, gone by 10–20 min, matching the literature. But most of the apparent signal is post-hoc and unusable in advance.
4. **Dekker's proportional shedding is mostly falsified** at junction scale. Delay change is additive between points; proportional only at terminals.
5. **Heavy tails demand intervals**, not point estimates.
6. **Weather adds nothing at junction scale** — and, more usefully, is ruled out as the confounder behind the knock-on correlation.

Next — the Phase 1 gate

Calibrate SUMO against January. Success means reproducing:

Per-station median delay curve	including the exit-station drop
`delay_gained` distribution	median $\approx +3$ s, mean $\approx +27$ s, p99 $\approx +350$ s
Headway distribution	median 5.3 min
Segment slopes	flat runs; terminal dwell -0.03 to -0.07
Model performance	MAE ≈56–60 / RMSE ≈92–99 on real test data

This is the project's hinge

If SUMO cannot reproduce the heavy tail, it is not a valid oracle and the programme falls back to observational-only. If simulated data makes the task markedly *easier*, the simulator is missing the noise that makes reality hard — that is a failure, not a success. **Find out in month one.**

Reproducing this

`data/extract_junction.py`	raw monthly → junction subset (≈7 s)
`data/build_traversals.py`	junction subset → per-traversal table
`analysis/phase0/step1_pairs.py`	leader/follower pair table
`analysis/phase0/step2_baselines.py`	task, split, baselines, GBDT
`analysis/phase0/step3_dekker.py`	exponential-decay test
`analysis/phase0/step4_figures.py`	all figures in this deck

Written notes: `data/README.md` (data), `literature/SOTA_NOTES.md` (state of the art, with fact-checks), `analysis/phase0/PHASE0.md` (Phase 0 results).

Figure palette: validated categorical slots 1–3, checked with the colour-vision-deficiency validator for all-pairs separation; aqua sits below 3:1 contrast on the light surface, so aqua marks carry direct labels.